Übungsblatt 3: Speicherzugriff und Kontrollfluss

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Einen Messpuffer ändern (10 Punkte)

RV32I, 32-Bit-Wörter, Little Endian. Ab `0x4800` liegt der beschreibbare Puffer $[9, -6, 14, 2]$. `s1=0x4808` zeigt bereits auf das dritte Wort.

```
lw  t0, -4(s1)
sub t1, x0, t0
sw  t1, 4(s1)
```

1. Geben Sie nach jeder Zeile effektive Adresse, Registeränderung und Speicheränderung an. (6 P)
2. Nennen Sie den vollständigen Puffer nach den drei Zeilen und das Datenquellregister des Stores. (2 P)
3. Welche vier Bytes stehen danach bei `0x480C`? Welchen Wert hat s1 am Ende? (2 P)

Musterlösung

Zeile	Adresse	Registeränderung	Speicheränderung
1	`0x4804`	`t0=0xFFFFFFFA` = −6	keine
2	kein Zugriff	`t1=6`	keine
3	`0x480C`	keine	Wort 2 wird 6

1. Je Zeile 2 P für vollständige Zustandsänderung.
2. $[9, -6, 14, 6]$ (1 P); t1 liefert die Store-Daten (1 P).
3. Bytes `06 00 00 00` (1 P); s1 bleibt `0x4808` (1 P). Der Offset ändert nicht das Basisregister.

Aufgabe 2: Zugriff vollständig im Bereich? (8 Punkte)

Nur `0x5000` bis einschließlich `0x500D` sind zugreifbar.

1. Bestimmen Sie effektive Adresse, natürliche Ausrichtung und Gültigkeit des **gesamten** Bytebereichs. Wort = vier, Halbwort = zwei Byte. (6 P)

Basis	Offset	Breite
`0x5001`	+7	Wort
`0x5001`	+4	Halbwort
`0x5002`	−2	Wort
`0x500B`	+1	Halbwort
`0x500D`	0	Wort
`0x500E`	0	Byte

2. Weshalb genügt ein gerader Offset nicht zur Halbwortausrichtung? Muss jeder unausgerichtete Zugriff in jeder RISC-V-Umgebung zwingend abbrechen? (2 P)

Musterlösung

Adresse	ausgerichtet?	ganzer Bereich gültig?
`0x5008`	ja	ja
`0x5005`	nein	ja
`0x5000`	ja	ja
`0x500C`	ja	ja
`0x500D`	nein	nein
`0x500E`	ja	nein

1. Je vollständiger Zeile 1 P, bei einzelnen richtigen Angaben je 1/3 P.
2. Basis plus Offset entscheidet; ungerade Basis plus gerader Offset bleibt ungerade (1 P). Die Behandlung unausgerichteter Zugriffe hängt von der Ausführungsumgebung ab; ein universeller Trap wird nicht vorausgesetzt (1 P).

Aufgabe 3: Ein Paket teilweise überschreiben (10 Punkte)

`s1=0x6000`. Anfangs liegen ab dort die Bytes `C3 7A A5 F1`, alle Zugriffe sind gültig.

1. Bestimmen Sie Registerwort und Zahlenwert für die vier Loads vor jeder Speicheränderung. (4 P)

```
lb  t0, 0(s1)
lbu t1, 0(s1)
lh  t2, 2(s1)
lhu t3, 0(s1)
```

2. Anschließend gilt `t4=0xDEADBEEF`. Notieren Sie die vier Bytes nach `sb t4, 0(s1)` und anschließend nach `sh t4, 2(s1)`. Welches 32-Bit-Wort ergibt sich zuletzt? (4 P)
3. Was passiert mit t4? Warum benötigen Stores keinen signed-/unsigned-Unterschied wie `lb` und `lbu`? (2 P)

Musterlösung

1. `t0=0xFFFFFFC3` = −61; `t1=0x000000C3` = 195; `t2=0xFFFFF1A5` = −3675; `t3=0x00007AC3` = 31427. Je Load 1 P, Wort und Zahl je 0,5.
2. Nach sb: `EF 7A A5 F1` (1,5 P). Nach sh: `EF 7A EF BE` (1,5 P). Letztes Wort: `0xBEEF7AEF` (1 P).
3. t4 bleibt unverändert (1 P). Stores übernehmen nur die unteren Quellbits in die festgelegte Speicherbreite; es findet keine Erweiterung oberer Zielbits statt (1 P).

Aufgabe 4: Store-Wörter zusammensetzen (10 Punkte)

S-Format: imm[11:5], rs2, rs1, funct3, imm[4:0], opcode; Breiten 7, 5, 5, 3, 5, 7 Bit. Für sw gelten funct3 = `010`, opcode = `0100011`. Aliase: a2 = x12, s0 = x8, t3 = x28, s2 = x18.

1. Codieren Sie `sw a2, -20(s0)`. Zeigen Sie das zwölfbittige Immediate, beide Teile, die sechs Felder und das Hexwort. (6 P)
2. Decodieren Sie `0x01C92E23` mit Offset und Aliasnamen. Warum enthält das Format kein rd? (4 P)

Musterlösung

1. −20: `0xFEC` = `111111101100`; oben `1111111`, unten `01100`. Felder: `1111111 01100 01000 010 01100 0100011`. Hexwort `0xFEC42623`. Immediate und Teilung 2 P, Register und Funktionsfelder 2 P, Hexwort 2 P.
2. Felder: `0000000 11100 10010 010 11100 0100011`. Offset 28, also `sw t3, 28(s2)`. Operation 1 P, Register 1 P, Offset 1 P. Kein rd: Der Store schreibt Speicher; die Positionen 11:7 sind Immediate-Bits (1 P).

Aufgabe 5: Das kleinere Wort auswählen (12 Punkte)

Der erste native Befehl liegt bei `0x2800`. Alle Instruktionen belegen vier Byte.

```
    blt  s0, s1, TakeA
    add  t0, s1, x0
    jal  x0, EndMin
TakeA:
    add  t0, s0, x0
EndMin:
    xori t1, t0, -1
```

1. Nennen Sie alle Befehls- und Labeladressen. (3 P)
2. Verfolgen Sie die ausgeführten Adressen und Endwerte t0, t1 für $s0 = -4, s1 = 6$ und $s0 = 9, s1 = 2$. (4 P)
3. Bestimmen Sie Branch- und Jump-Byteoffset. (2 P)
4. Ersetzen Sie nur blt durch bltu. Welcher Pfad und welcher t0-Wert entstehen für $(-4, 6)$? (2 P)
5. Der Jump fehlt in einer fehlerhaften Variante. Warum entsteht für $(9, 2)$ ein falsches Minimum? Eine neue Adressrechnung ist nicht verlangt. (1 P)

Musterlösung

1. Befehle `2800`, `2804`, `2808`, `280C`, `2810`; TakeA = `0x280C`, EndMin = `0x2810`. Befehlsadressen 2 P, Labels zusammen 1 P.
2. $(-4, 6)$: `2800` → `280C` → `2810`, danach `2814`; t0 = −4, t1 = 3. $(9, 2)$: `2800` → `2804` → `2808` → `2810`, danach `2814`; t0 = 2, t1 = −3. Je Fall Pfad 1 P und Endwerte 1 P.
3. Branch +12 Byte; Jump +8 Byte. Je 1 P; Ausgangspunkt ist jeweils die Adresse der Sprunginstruktion.
4. Unsigned ist `0xFFFFFFFC` größer als 6: Branch nicht genommen, t0 wird 6. Je 1 P.
5. Der Pfad schreibt zunächst 2, fällt dann aber in TakeA und überschreibt t0 mit 9. 1 P.

Aufgabe 6: Negative Elemente zählen (10 Punkte)

Array $[-7, 2, 0, -3]$ ab `B=0x7000`. Anfang `t0=0`, `t1=4`, `s0=0`, `s1=B`; t2 ist unbekannt.

```
Loop:
    bgeu t0, t1, Done
    lw   t2, 0(s1)
    bge  t2, x0, Next
    addi s0, s0, 1
Next:
    addi s1, s1, 4
    addi t0, t0, 1
    jal  x0, Loop
Done:
```

1. Notieren Sie vor jedem bgeu einschließlich Endtest: t0, s1, s0 und die Branch-Entscheidung. (5 P)
2. Nennen Sie Load- und Endtest-Anzahl sowie das letzte t2-Wort. (2 P)
3. Wiederholen Sie gedanklich mit t1 = 0. Welche Werte bleiben in s0, s1 und t2? (2 P)
4. Welche einzelne Branch-Änderung zählt stattdessen ausschließlich Nullen? (1 P)

Musterlösung

Test	t0	s1	s0	bgeu genommen?
1	0	`0x7000`	0	nein
2	1	`0x7004`	1	nein
3	2	`0x7008`	1	nein
4	3	`0x700C`	1	nein
5	4	`0x7010`	2	ja

1. Je vollständiger Zeile 1 P.
2. Vier Loads, fünf Endbedingungstests (zusammen 1 P); letzter t2 = `0xFFFFFFFD` (1 P).
3. s0 = 0 und s1 = B (zusammen 1 P); kein Load, t2 bleibt unbekannt und wird nicht benutzt (1 P).
4. `bne t2, x0, Next` überspringt das Zählen aller Nichtnullwerte. 1 P.

Aufgabe 7: Negative Messwerte in Venus zählen

Schreiben Sie ein vollständiges Programm für die ersten N Werte aus `.word -5, 8, -2, 7, -1, 0`. Es soll nur die Anzahl negativer Werte ausgeben, die Daten bleiben unverändert.

1. Initialisieren Sie Basis, Zähler, Länge und Ergebnis. Prüfen Sie die Länge vor jedem Load; $0 \leq N \leq 6$.
2. Prüfen Sie N = 6, 0 und 1 sowie den anderen Datensatz $[0, 2, 9, 4]$ mit N = 4.
3. Notieren Sie Ergebnis, Load-Anzahl und Pointerabstand zur tatsächlichen Basis vor der Ausgabe.
4. Venus-Dienste: Ausgabe mit a0 = 1, Zahl in a1; Ende mit a0 = 10. Assemblieren und starten Sie jede Variante neu.

Vergleichen Sie anschließend mit `beispiele/lab_03_negative_zaehlen.s`.

Musterlösung

Der Rumpf entspricht der Zählaufgabe, nicht der Arraysumme aus den Folien. Die neue Referenzdatei enthält einen vollständigen Venus-Rahmen.

Fall	Anzahl negativ	Loads	Pointerabstand
N = 6	3	6	24 Byte
N = 0	0	0	0 Byte
N = 1	1	1	4 Byte
anderer Datensatz, N = 4	0	4	16 Byte

Die jeweiligen Schleifentests zählen N + 1. Das sind Modellvorhersagen; echte Registeranzeigen und Ausgaben sind im eigenen Versuch zu protokollieren.